

3-Month AI Engineer Roadmap

From Senior Performance Test Engineer → Hireable AI Engineer

Starting point: Basic Python | **Commitment assumed:** 2–3 hrs/day weekdays + 4–5 hrs weekend (~20 hrs/week) **Philosophy:** Every week ends with a *shipped, working project on GitHub* — not just notebooks. Recruiters and hiring managers scan your GitHub first; make it look like production work (README, requirements.txt, tests, a live demo link where possible).

How to use this plan

1. Each week = **Learn → Build → Document → Push to GitHub**.
 2. Every project must have: a clean README (problem, approach, results, how to run), a `requirements.txt`/`pyproject.toml`, and ideally a Streamlit/Gradio demo or API endpoint.
 3. Keep ONE GitHub repo called `ai-engineer-journey` with sub-folders per week, and separately fork out your 3 best projects into **standalone repos** — those are what you link on your resume/LinkedIn.
 4. Write a short LinkedIn post every week (2–3 lines: what you built, what you learned). Recruiters see consistency.
-

MONTH 1 — Foundations (Python for Data, Math, Core ML)

Week 1: Python Data Stack (Pandas, NumPy, Matplotlib, Seaborn)

Learn:

- NumPy: arrays, broadcasting, vectorization, linear algebra ops
- Pandas: dataframes, groupby, merge/join, pivot tables, missing data, time series
- Matplotlib/Seaborn: distribution plots, correlation heatmaps, subplots, styling for reports

GitHub repos to study/practice:

- `rougier/numpy-100` — 100 NumPy exercises (do all of them)
- `guipsamora/pandas_exercises` — Pandas practice problems
- `pandas-dev/pandas` — skim source/docs for idioms
- `mwaskom/seaborn` — official example gallery (in `examples/`)
- `matplotlib/cheatsheets` — official cheat sheets

Project (ship it): "Performance Testing Analytics Dashboard" — since you're already a performance test engineer, take real/synthetic JMeter/Gatling logs (response time, throughput, error rate) and build a EDA notebook + Seaborn dashboard: percentile latency trends, anomaly detection on spikes, correlation between load and error rate. This project doubles as a **portfolio bridge from your current role to AI/data work** — very strong story for interviews.

Week 2: Math for ML (Linear Algebra + Statistics/Probability)

Learn:

- Linear Algebra: vectors, matrices, dot product, eigenvalues/eigenvectors, matrix decomposition (why it matters: embeddings, PCA, neural net weights)
- Statistics: mean/median/variance, distributions (normal, binomial), hypothesis testing, p-values, confidence intervals, correlation vs causation
- Probability: Bayes theorem, conditional probability (foundation for Naive Bayes, LLM token probabilities later)

GitHub repos:

- `fastai/numerical-linear-algebra` — practical linear algebra for coders (Jupyter notebooks)
- `ageron/hands-on-ml3` — has a math appendix + code notebooks (you'll reuse this repo again in Week 3-4)
- `khanacademy` has no repo, but pair this week with 3Blue1Brown's "Essence of Linear Algebra" playlist (visual intuition) alongside coding the concepts in NumPy

Project (ship it): Implement from scratch (NumPy only, no sklearn) — mean/variance/std, Pearson correlation, matrix multiplication, PCA using eigen-decomposition. Push as `math-for-ml-from-scratch` repo with a notebook explaining each formula next to its code — this signals real understanding, not just API calls, which interviewers probe for.

Week 3: ML Fundamentals — Part 1 (Supervised Learning)

Learn:

- Regression (Linear, Logistic), classification metrics (precision/recall/F1/ROC-AUC — you'll relate this to false positive/negative in test automation)
- Decision Trees, Random Forest, Gradient Boosting (XGBoost/LightGBM)
- Bias-variance tradeoff, overfitting/underfitting, cross-validation, train/test/val split

GitHub repos:

- [eriklindernoren/ML-From-Scratch](#) — implements every classic algorithm in raw Python/NumPy (excellent for interviews where they ask "how does X work")
- [ageron/handson-ml3](#) — companion code for "Hands-On ML with Scikit-Learn, Keras & TensorFlow" (the single best applied ML book/repo combo)
- [josephmisiti/awesome-machine-learning](#) — curated list, use as reference library

Project (ship it): "Predict flaky/failing test cases" — using historical test execution data (pass/fail, duration, environment, flakiness history), build a classifier to predict test failure probability before a run. This is a **genuinely useful enterprise use case** tying your QA background to ML — great interview talking point.

Week 4: ML Fundamentals — Part 2 + Frameworks (Scikit-learn)

Learn:

- Unsupervised: K-Means, hierarchical clustering, DBSCAN, PCA/t-SNE for dimensionality reduction
- Model selection: GridSearchCV/RandomizedSearchCV, pipelines, feature engineering, feature scaling/encoding
- Scikit-learn as a framework: [Pipeline](#), [ColumnTransformer](#), custom transformers
- Model persistence ([joblib](#)/[pickle](#)) and basic serving

GitHub repos:

- [scikit-learn/scikit-learn](#) — browse [examples/](#) folder directly
- [ageron/handson-ml3](#) (continue) — clustering & dimensionality reduction chapters
- [krishnaik06/Complete-Machine-Learning](#) — full applied ML project structures

Project (ship it): End-to-end ML pipeline: data ingestion → cleaning → feature engineering → model training with [Pipeline](#) → hyperparameter tuning → export model → wrap in a **FastAPI** endpoint → simple Streamlit UI. Use a public dataset (e.g., customer churn) if you don't have a better one. This is your first "productionized" project — structure it like a real repo: [src/](#), [tests/](#), [Dockerfile](#), [README.md](#).

Month 1 Checkpoint: You should now have 4 repos on GitHub, comfortable explaining bias-variance, precision/recall, and be able to build+deploy a classical ML model end-to-end.

MONTH 2 — Deep Learning & GenAI Core

Week 5: Deep Learning Fundamentals

Learn:

- Perceptron → MLP, forward/backward propagation, activation functions, loss functions, gradient descent variants (SGD, Adam)
- Regularization: dropout, batch norm, early stopping
- PyTorch basics (tensors, autograd, `nn.Module`, training loop) — learn PyTorch first even though your target is TensorFlow, because most modern GenAI research/repos are PyTorch-native

GitHub repos:

- `pytorch/examples` — official minimal examples (MNIST, etc.)
- `aladdinpersson/Machine-Learning-Collection` — PyTorch implementations of everything from basics to advanced architectures
- `karpathy/micrograd` — build a tiny autograd engine from scratch (does wonders for real understanding of backprop)

Project (ship it): Build a neural network **from scratch in NumPy** (no framework) to classify MNIST digits — then rebuild the same model in PyTorch in 10 lines. Push both versions side-by-side in one repo (`neural-net-from-scratch-vs-pytorch`) — this is a classic "prove you understand fundamentals" interview project.

Week 6: Deep Learning — CNNs, RNNs, Transformers (Conceptual)

Learn:

- CNNs: convolution, pooling, image classification (why they matter even for GenAI: vision-language models)
- RNN/LSTM basics (sequence modeling) — enough to understand *why* Transformers replaced them
- Transformer architecture: self-attention, multi-head attention, positional encoding — **this is the single most important concept for your GenAI phase**

GitHub repos:

- `karpathy/nanoGPT` — the best repo to understand transformers by building GPT from scratch
- `jadore801120/attention-is-all-you-need-pytorch` — Transformer paper implementation
- `aladdinpersson/Machine-Learning-Collection` (continue — has CNN/RNN sections)

Project (ship it): Follow `karpathy/nanoGPT` and train a tiny character-level GPT on a small text corpus (e.g., Shakespeare). Document the attention mechanism with your own diagrams in the README. This single project is a huge signal to interviewers that you

understand what's actually happening inside an LLM — most "prompt engineers" cannot do this.

Week 7: TensorFlow / Keras

Learn:

- TensorFlow 2.x + Keras API: Sequential/Functional API, custom training loops, `tf.data` pipelines
- Transfer learning (fine-tune a pretrained CNN, e.g., ResNet/MobileNet)
- TensorFlow Serving / TF Lite basics (deployment awareness)
- Rebuild your Week 5/6 projects in TensorFlow so you're fluent in both major frameworks (many enterprise JDs still ask TensorFlow specifically)

GitHub repos:

- `tensorflow/tensorflow` — official repo, browse `tensorflow/docs` tutorials folder
- `aymericdamien/TensorFlow-Examples` — from basics to advanced, TF2-updated
- `tensorflow/models` — official model garden for transfer learning examples

Project (ship it): Image classification with transfer learning (Keras + pretrained model) deployed via a Streamlit app with drag-and-drop image upload + live prediction + confidence scores. Deploy it publicly (Hugging Face Spaces or Streamlit Cloud) so you have a **live demo link** for your resume.

Month 2 Checkpoint: You understand neural nets from scratch, can explain transformer attention, and are fluent in both PyTorch and TensorFlow. This is exactly the technical bar product companies test for in ML/AI engineer interviews.

Week 8: Generative AI Fundamentals

Learn:

- LLM landscape: GPT/Claude/Llama family, tokenization, context windows, temperature/top-p sampling
- Prompt engineering: zero-shot, few-shot, chain-of-thought, structured output prompting
- Using LLM APIs (OpenAI/Anthropic/open-source via Hugging Face) programmatically
- Fine-tuning concepts: full fine-tune vs LoRA/QLoRA (parameter-efficient fine-tuning) — don't need to master training yet, just understand the landscape

GitHub repos:

- [microsoft/generative-ai-for-beginners](#) — structured 18-lesson curriculum with code
- [huggingface/transformers](#) — browse [examples/](#) for pipeline usage
- [openai/openai-cookbook](#) — official patterns for prompting, embeddings, function calling
- [anthropics/anthropic-cookbook](#) — Claude-specific prompting & tool-use patterns (directly useful since you'll likely work with Claude API too)

Project (ship it): Build a "Test Case Generator" — feed a requirement/user story to an LLM API and generate structured test cases (positive/negative/edge cases) as JSON, then auto-convert to a test management format (CSV/Excel). Directly leverages your QA domain expertise + GenAI — a genuinely differentiated portfolio piece nobody else in a generic bootcamp will have.

MONTH 3 — Applied GenAI Engineering (RAG, LangChain, Agents)

Week 9: Vector Databases & Embeddings

Learn:

- What embeddings are, embedding models (OpenAI, [sentence-transformers](#)), cosine similarity
- Vector DB concepts: indexing (HNSW, IVF), similarity search, hybrid search, metadata filtering
- Hands-on with FAISS (local), Chroma (lightweight), Qdrant/Pinecone/Weaviate (production-grade)

GitHub repos:

- [facebookresearch/faiss](#) — the foundational similarity search library
- [chroma-core/chroma](#) — easiest vector DB to start with locally
- [qdrant/qdrant](#) — production-grade, good docs/examples for a "real" system
- [UKPLab/sentence-transformers](#) — for generating embeddings locally (no API cost)

Project (ship it): Build a **semantic search engine over your company's/your own test documentation or requirement docs** — embed documents, store in Chroma/Qdrant, build a search UI that returns semantically relevant docs (not just keyword match). This is the direct precursor skill to RAG.

Week 10: RAG (Retrieval-Augmented Generation)

Learn:

- RAG architecture end-to-end: chunking strategies, embedding, retrieval, reranking, prompt assembly, generation
- Chunking strategies (fixed-size, semantic, recursive), overlap tuning
- Evaluation of RAG systems: faithfulness, relevance, context precision/recall (RAGAS framework)
- Advanced RAG: hybrid search, re-ranking (cross-encoders), query rewriting/expansion

GitHub repos:

- [NirDiamant/RAG_Techniques](#) — one of the best repos covering 20+ RAG techniques with code, from naive to advanced
- [langchain-ai/rag-from-scratch](#) — official LangChain notebooks building RAG piece by piece
- [explodinggradients/ragas](#) — for evaluating your RAG pipeline quality (very impressive to mention in interviews)

Project (ship it): "Enterprise Document Q&A System" — build a full RAG pipeline (no framework yet, raw Python + FAISS/Chroma + LLM API) over a real document set (e.g., your organization's public policy docs, or any large PDF corpus like a technical manual). Include: chunking strategy comparison, retrieval evaluation with RAGAS, and a Streamlit chat UI. This is the **single most commonly asked "build me a RAG system" interview/take-home task** — having it done in advance is huge leverage.

Week 11: LangChain (and intro to LangGraph)

Learn:

- LangChain core: [Chains](#), [Prompts](#), [Memory](#), [Retrievers](#), [Tools](#), [Output Parsers](#), LCEL (LangChain Expression Language)
- Rebuild your Week 10 RAG project using LangChain to see how much boilerplate it removes
- Intro to LangGraph: stateful, graph-based orchestration (this bridges into agents next week)
- LangSmith basics for tracing/debugging LLM chains (production observability — big enterprise signal)

GitHub repos:

- [langchain-ai/langchain](#) — official repo, study [cookbook/](#) and [templates/](#)
- [langchain-ai/langgraph](#) — official repo + examples folder
- [gkamradt/langchain-tutorials](#) — practical community tutorials, well-organized

Project (ship it): Refactor your RAG project into a LangChain-based **multi-document chatbot with conversational memory** (follow-up questions, source citations shown in UI). Add LangSmith tracing so you can show a screenshot of a debugged trace in your README — that's a very senior-level touch.

Week 12: AI Agents (Capstone Month)

Learn:

- Agent concepts: ReAct pattern, tool calling/function calling, planning, multi-step reasoning
- Multi-agent orchestration: LangGraph (stateful agent graphs), CrewAI (role-based agent teams), AutoGen (conversational multi-agent)
- Agent memory, guardrails, and evaluation; when to use agents vs plain RAG (cost/latency/reliability tradeoffs — important for interviews, shows maturity)

GitHub repos:

- `langchain-ai/langgraph` (continue) — build a stateful agent with tool use and loops
- `NirDiamant/GenAI_Agents` — 20+ agent patterns/tutorials with code, excellent breadth
- `crewAIInc/crewAI` — role-based multi-agent framework, very readable examples
- `microsoft/autogen` — Microsoft's multi-agent conversation framework

Capstone Project (this is your flagship resume project): "Autonomous QA Agent" — combining everything you've built this quarter:

- An agent that takes a requirement/user story as input
- Uses a **tool** to generate test cases (LLM call)
- Uses **RAG** to retrieve similar past defects/test data from a vector DB
- Uses another **tool** to execute/simulate test runs and log results
- A **planner agent** decides next steps (retest, escalate, mark flaky) based on results
- Built with LangGraph or CrewAI, deployed with a Streamlit/FastAPI interface, fully documented, with architecture diagram in the README, and ideally deployed publicly (Hugging Face Spaces / Render / Railway)

This project is powerful specifically *because* it's grounded in your real domain expertise (performance/QA testing) — it tells a hiring manager "this person can apply GenAI to solve a real enterprise problem," which is exactly what product companies pay for, versus a generic "chatbot over PDF" clone everyone else has.

Resume & Job-Search Layer (run in parallel from Week 8 onward)

- Pin your **top 3 projects** on GitHub profile: (1) Flaky-test predictor (classical ML), (2) Enterprise RAG Q&A system, (3) Autonomous QA Agent (capstone)
- Each project repo needs: problem statement, architecture diagram, tech stack badges, setup instructions, and (ideally) a live demo link or a short screen-recording GIF
- Write a 1-page case study for the capstone project (what problem, why RAG/agents, tradeoffs considered, results) — this becomes your interview narrative
- Target roles: "AI Engineer," "GenAI Engineer," "ML Engineer," "LLM Application Engineer" — your performance-testing background is a genuine differentiator, use it explicitly in your resume summary
- Practice explaining every project's *design decisions* out loud (chunking size, why this vector DB, why this agent framework) — product companies interview for reasoning, not just output

Quick Reference: All Repos by Topic

Topic	Repos
Pandas/NumPy/Viz	rougier/numpy-100 , guipsamora/pandas_exercises , mwaskom/seaborn
Math	fastai/numerical-linear-algebra , ageron/handson-ml3
ML Fundamentals	eriklindernoren/ML-From-Scratch , ageron/handson-ml3 , josephmisiti/awesome-machine-learning
Frameworks (sklearn)	scikit-learn/scikit-learn , krishnaik06/Complete-Machine-Learning
Deep Learning	pytorch/examples , aladdinpersson/Machine-Learning-Collection , karpathy/micrograd , karpathy/nanoGPT
TensorFlow	tensorflow/tensorflow , aymericdamien/TensorFlow-Examples , tensorflow/models
GenAI	microsoft/generative-ai-for-beginners , openai/openai-cookbook , anthropics/anthropic-cookbook
Vector DB	facebookresearch/faiss , chroma-core/chroma , qdrant/qdrant , UKPLab/sentence-transformers
RAG	NirDiamant/RAG_Techniques , langchain-ai/rag-from-scratch , explodinggradients/ragas

Topic	Repos
LangChain	<code>langchain-ai/langchain</code> , <code>langchain-ai/langgraph</code> , <code>gkamradt/langchain-tutorials</code>
AI Agents	<code>NirDiamant/GenAI_Agents</code> , <code>crewAIInc/crewAI</code> , <code>microsoft/autogen</code>

Project Ship List (12 weeks → 12 shippable artifacts)

1. Performance testing analytics dashboard
2. Math-for-ML-from-scratch notebook
3. Flaky test predictor (classifier)
4. End-to-end ML pipeline + FastAPI + Streamlit
5. NumPy vs PyTorch neural net comparison
6. nanoGPT-style tiny transformer
7. Image classifier with transfer learning (deployed)
8. LLM-based test case generator
9. Semantic search over docs (vector DB)
10. Enterprise RAG Q&A system (with RAGAS eval)
11. LangChain conversational RAG chatbot (with LangSmith tracing)
12. **Capstone: Autonomous QA Agent** (multi-agent + RAG + tools)